Лабораторная работа: "Объектно-ориентированная реализация структур данных"

Цель работы

Отработка тем по ООП (поля и методы класса, конструкторы, наследование, полиморфизм, перегрузка операторов), работа с исключениями, создание библиотек классов, реализация базовых структур данных.

Теоретическая часть

Структуры данных, рассматриваемые в работе:

1. **Односвязный список** - структура, где каждый элемент (узел) содержит данные и ссылку на следующий элемент
2. **Двусвязный список** - каждый узел содержит ссылки на следующий и предыдущий элементы
3. **Стек** - структура с принципом LIFO (Last In, First Out)
4. **Очередь** - структура с принципом FIFO (First In, First Out)

Основные принципы ООП, применяемые в работе:

- **Инкапсуляция** - объединение данных и методов в классах
- **Наследование** - создание новых классов на основе существующих
- **Полиморфизм** - переопределение методов в наследниках
- **Перегрузка операторов** - определение поведения стандартных операторов

Задание на лабораторную работу

Часть 1. Базовый класс Node (Узел)

Создать класс Node, представляющий узел структуры данных:

Поля класса:

- data - данные, хранящиеся в узле
- next - ссылка на следующий узел (по умолчанию None)

Методы:

1. Конструктор __init__ должен поддерживать три режима работы:
    - Без параметров: создает узел с data = 0 и next = None
    - С одним параметром: Node(5) - создает узел с данными 5 и next = None
    - С двумя параметрами: Node(5, next_node) - создает узел с данными 5 и ссылкой на следующий узел
2. Метод clear() - обнуляет данные (устанавливает в 0) и убирает ссылку на следующий узел (устанавливает None)
3. Переопределить метод __str__ для строкового представления узла (например, "Node(5)")

Часть 2. Односвязный список (SinglyLinkedList)

Создать класс SinglyLinkedList, который **использует композицию** с классом Node (содержит узлы, но не наследуется от Node).

Поля класса:

- head - ссылка на первый узел списка (None для пустого списка)
- size - количество элементов в списке (опционально)

Методы добавления элементов:

1. prepend(data) - добавление элемента в начало списка
2. append(data) - добавление элемента в конец списка
3. insert_after(index, data) - добавление элемента после элемента с указанным индексом
4. insert_before(index, data) - добавление элемента перед элементом с указанным индексом

Методы удаления элементов:

1. remove_first() - удаление первого элемента
2. remove_last() - удаление последнего элемента
3. remove_at(index) - удаление элемента по индексу
4. remove_value(value) - удаление первого вхождения указанного значения

Методы поиска элементов:

1. find(value) - поиск индекса первого вхождения значения (возвращает индекс или -1)
2. find_all(value) - поиск индексов всех вхождений значения (возвращает список индексов)
3. contains(value) - проверка наличия значения в списке (возвращает True/False)
4. get(index) - получение значения элемента по индексу

Дополнительные методы:

1. __len__ - возвращает количество элементов
2. __str__ - строковое представление списка (например, "1 -> 2 -> 3 -> None")
3. __iter__ - сделать список итерируемым
4. clear() - очистка списка

Часть 3. Двусвязный список (DoublyLinkedList)

Создать класс DoublyLinkedList, используя **наследование** для узла:

Сначала создать класс DoubleNode, наследующий от Node:

```
class DoubleNode(Node):
    def __init__(self, data=0, next=None, prev=None):
        super().__init__(data, next)
        self.prev = prev  # ссылка на предыдущий узел
```

Затем создать класс DoublyLinkedList.

Поля класса:

- head - ссылка на первый узел
- tail - ссылка на последний узел (опционально)

Методы (аналогично односвязному списку, но с учетом двусвязности):

- Добавление: prepend, append, insert_after, insert_before
- Удаление: remove_first, remove_last, remove_at, remove_value
- Поиск: find, find_all, contains, get
- Дополнительные: __len__, __str__, __iter__, clear

Особенности для двусвязного списка:

- При добавлении/удалении нужно обновлять обе ссылки (next и prev)
- Можно реализовать проход по списку как с начала, так и с конца

Часть 4. Стек (Stack)

Создать класс Stack на основе односвязного списка (использовать композицию).

Поля класса:

- items - объект класса SinglyLinkedList (или своя реализация)

Методы:

1. push(data) - добавить элемент на вершину стека
2. pop() - удалить и вернуть элемент с вершины стека
3. peek() - посмотреть элемент на вершине без удаления
4. is_empty() - проверка, пуст ли стек
5. size() - количество элементов
6. from_queue(queue) - **статический метод**, создающий новый стек из очереди (элементы из очереди переносятся в стек)

Часть 5. Очередь (Queue)

Создать класс Queue на основе односвязного списка.

Поля класса:

- items - объект класса SinglyLinkedList

Методы:

1. enqueue(data) - добавить элемент в конец очереди
2. dequeue() - удалить и вернуть элемент из начала очереди
3. front() - посмотреть первый элемент без удаления
4. is_empty() - проверка, пуста ли очередь
5. size() - количество элементов

6. from_stack(stack) - **статический метод**, создающий новую очередь из стека (элементы из стека переносятся в очередь)

Часть 6. Система исключений

Создать иерархию классов для обработки ошибок, специфичных для структур данных:

```
class StructureError(Exception):
    """Базовый класс для всех исключений структур данных"""
    pass


class EmptyStructureError(StructureError):
    """Ошибка при попытке операции с пустой структурой"""
    def __init__(self, structure_name, message="Structure is empty"):
        self.structure_name = structure_name
        super().__init__(f"{message}: {structure_name}")


class IndexError(StructureError):
    """Ошибка при обращении по несуществующему индексу"""
    def __init__(self, index, message="Index out of range"):
        self.index = index
        super().__init__(f"{message}: {index}")


class ValueNotFoundError(StructureError):
    """Ошибка при поиске несуществующего значения"""
    def __init__(self, value, message="Value not found"):
        self.value = value
        super().__init__(f"{message}: {value}")
```

Часть 7. Перегрузка операторов

В классах списков перегрузить следующие операторы:

- __getitem__ - обращение по индексу (list[0])
- __setitem__ - изменение по индексу (list[0] = 10)
- __contains__ - оператор in
- __add__ - конкатенация двух списков
- __mul__ - повторение списка (list * 3)

Часть 8. Библиотека классов

1. Все созданные классы вынести в отдельный модуль (например, structures.py)
2. Модуль должен быть организован так, чтобы его можно было импортировать:
3. from structures import SinglyLinkedList, Stack, Queue, StructureError

Часть 9. Демонстрационная программа

Создать основную программу (main.py), которая демонстрирует:

1. **Создание объектов** всех типов
2. **Добавление элементов** всеми способами
3. **Поиск элементов**:
4. lst = SinglyLinkedList()
5. lst.append(10)
6. lst.append(20)
7. lst.append(30)
8. lst.append(20)
9.
10. print(lst.find(20)) # 1
11. print(lst.find_all(20)) # [1, 3]
12. print(20 in lst) # True
13. **Удаление элементов**
14. **Работу стеков и очередей**
15. **Преобразование между структурами**:
16. queue = Queue()
17. queue.enqueue(1)
18. queue.enqueue(2)
19.
20. stack = Stack.from_queue(queue) # из очереди в стек
21. new_queue = Queue.from_stack(stack) # из стека в очередь
22. **Обработку исключений**:
23. try:
24. lst = SinglyLinkedList()
25. elem = lst.get(0) # пустой список
26. except IndexError as e:
27. print(f"Ошибка: {e}")

28. **Работу перегруженных операторов**:
29. lst1 = SinglyLinkedList()
30. lst1.append(1)
31. lst1.append(2)
32.
33. lst2 = SinglyLinkedList()
34. lst2.append(3)
35. lst2.append(4)
36.
37. lst3 = lst1 + lst2 # конкатенация
38. print(lst3[0]) # 1
39. lst3[0] = 10   # изменение

Требования к оформлению

1. Код должен быть хорошо документирован (docstrings для классов и методов)
2. Соблюдение стиля PEP 8
3. Все классы должны быть в отдельном файле (библиотеке)
4. Демонстрационная программа должна показывать работу всех методов
5. Обязательно наличие обработки исключений

Пример ожидаемого вывода

Демонстрация работы односвязного списка:

Создаем список: []

Добавляем в начало: 1 -> None

Добавляем в конец: 1 -> 2 -> None

Вставляем после индекса 1 (значение 3): 1 -> 2 -> 3 -> None

Вставляем перед индексом 1 (значение 4): 1 -> 4 -> 2 -> 3 -> None

Поиск элементов:

Индекс значения 2: 2

Все индексы значения 1: [0, ?]

Проверка наличия 5: False

Удаление элемента с индекса 2: 1 -> 4 -> 3 -> None

Демонстрация работы стека:

Добавили 5, 10, 15

Верхний элемент: 15

Удалили: 15

Текущий стек: [5, 10]

Демонстрация преобразования:

Очередь: [1, 2, 3]

Стек из очереди: [3, 2, 1]

Обработка исключений:

Попытка получить элемент из пустого списка...

Ошибка: Index out of range: 0

Методические указания

1. Начните с реализации класса Node
2. Реализуйте односвязный список, тщательно тестируя каждую операцию
3. Добавьте обработку граничных случаев (пустой список, неверный индекс)
4. Перейдите к двусвязному списку, используя наследование
5. Реализуйте стек и очередь как обертки над списком
6. Добавьте систему исключений
7. В конце реализуйте перегрузку операторов